

Referencia Rápida: Operaciones en Listas Simplemente Enlazadas

Programación II

Abril 2024

Introducción

Este documento contiene las operaciones fundamentales y avanzadas para el manejo de listas simplemente enlazadas en C++. Incluye complejidades, parámetros y ejemplos de código.

1 Representación Base

```
1 struct nodoLista {  
2     Tipo info; // 'Tipo' es el tipo de dato almacenado  
3     nodoLista *sig;  
4 };  
5 typedef nodoLista* Lista; // 'Lista' es un puntero al primer nodo
```

Nota: Una lista vacía se representa con un puntero NULL.

2 Operaciones Fundamentales

2.1 Crear Lista Vacía

Propósito: Inicializar una lista sin elementos.

```
1 Lista crearListaVacía() {  
2     return NULL;  
3 }
```

Complejidad: $O(1)$

Parámetros: No aplica

2.2 Verificar si está Vacía

Propósito: Determinar si la lista contiene elementos.

```
1 bool esVacía(Lista l) {  
2     return (l == NULL);  
3 }
```

Complejidad: $O(1)$

Parámetros: Por valor (Lista l)

2.3 Insertar al Comienzo

Propósito: Añadir elemento al inicio.

```
1 void insComienzo(Lista &l, Tipo e) {
2     Lista nuevo = new nodoLista;
3     nuevo->info = e;
4     nuevo->sig = l;
5     l = nuevo;
6 }
```

Complejidad: $O(1)$

Parámetros: Por referencia (&l), valor (e)

2.4 Recorrer/Imprimir

Propósito: Visitar cada nodo (ejemplo de impresión).

```
1 void imprimirLista(Lista l) {
2     Lista aux = l;
3     while (aux != NULL) {
4         std::cout << aux->info << " -> ";
5         aux = aux->sig;
6     }
7     std::cout << "NULL" << std::endl;
8 }
```

Complejidad: $O(n)$

Parámetros: Por valor (Lista l)

2.5 Buscar Elemento

Propósito: Determinar si un elemento existe.

```
1 bool buscarElemento(Lista l, Tipo e) {
2     Lista aux = l;
3     while (aux != NULL && aux->info != e) {
4         aux = aux->sig;
5     }
6     return (aux != NULL);
7 }
```

Complejidad: $O(n)$

Parámetros: Por valor (l, e)

2.6 Insertar al Comienzo

Propósito: Añadir elemento al inicio.

```
1 void insComienzo(Lista &l, Tipo e) {
2     Lista nuevo = new nodoLista;
3     nuevo->info = e;
4     nuevo->sig = l;
5     l = nuevo;
6 }
```

Complejidad: $O(1)$

Parámetros: Por referencia (&l), valor (e)

2.7 Recorrer/Imprimir

Propósito: Visitar cada nodo (ejemplo de impresión).

```
1 void imprimirLista(Lista l) {  
2     Lista aux = l;  
3     while (aux != NULL) {  
4         std::cout << aux->info << " -> ";  
5         aux = aux->sig;  
6     }  
7     std::cout << "NULL" << std::endl;  
8 }
```

Complejidad: $O(n)$

Parámetros: Por valor (Lista l)

2.8 Buscar Elemento

Propósito: Determinar si un elemento existe.

```
1 bool buscarElemento(Lista l, Tipo e) {  
2     Lista aux = l;  
3     while (aux != NULL && aux->info != e) {  
4         aux = aux->sig;  
5     }  
6     return (aux != NULL);  
7 }
```

Complejidad: $O(n)$

Parámetros: Por valor (l, e)

2.9 Borrar Primer Elemento

Propósito: Eliminar el nodo inicial.

```
1 void elimComienzo(Lista &l) {  
2     if (l != NULL) {  
3         Lista aBorrar = l;  
4         l = l->sig;  
5         delete aBorrar;  
6     }  
7 }
```

Complejidad: $O(1)$

Parámetros: Por referencia (&l)

2.10 Insertar al Final (Iterativo)

Propósito: Añadir elemento al final.

```
1 void insFinal(Lista &l, Tipo e) {
2     Lista nuevo = new nodoLista;
3     nuevo->info = e;
4     nuevo->sig = NULL;
5
6     if (l == NULL) {
7         l = nuevo;
8     } else {
9         Lista aux = l;
10        while (aux->sig != NULL) {
11            aux = aux->sig;
12        }
13        aux->sig = nuevo;
14    }
15 }
```

Complejidad: $O(n)$

Parámetros: Por referencia (&l), valor (e)

2.11 Inserción Ordenada (Recursiva)

Propósito: Insertar manteniendo orden.

```
1 void insOrd(Lista &l, Tipo e) {
2     if (l == NULL || e <= l->info) {
3         insComienzo(l, e);
4     } else {
5         insOrd(l->sig, e);
6     }
7 }
```

Complejidad: $O(n)$

Parámetros: Por referencia (&l), valor (e)

3 Operaciones Avanzadas

3.1 Merge (Intercalación Ordenada)

Propósito: Combinar dos listas ordenadas.

```
1 Lista merge(Lista l1, Lista l2) {
2     Lista res = NULL;
3     Lista ult = NULL;
4
5     while (l1 != NULL && l2 != NULL) {
6         if (l1->info < l2->info) {
7             insFinalAux(res, ult, l1->info);
8             l1 = l1->sig;
9         } else {
10            insFinalAux(res, ult, l2->info);
11            l2 = l2->sig;
12        }
13    }
```

```

13     }
14     // A adir restantes...
15     return res;
16 }

```

3.2 Invertir Lista (Reusando Nodos)

Propósito: Invertir lista modificando punteros.

```

1 Lista invListaReusaNodos(Lista l) {
2     Lista l_inv = NULL;
3     Lista actual = l;
4     Lista siguiente;
5
6     while (actual != NULL) {
7         siguiente = actual->sig;
8         actual->sig = l_inv;
9         l_inv = actual;
10        actual = siguiente;
11    }
12    return l_inv;
13 }

```

3.3 Borrar Último Elemento (Iterativo)

Propósito: Eliminar el nodo final de la lista.

```

1 void elimUltimo(Lista &l) {
2     if (l == NULL) return;
3
4     if (l->sig == NULL) {
5         delete l;
6         l = NULL;
7     } else {
8         Lista anterior = l;
9         while (anterior->sig->sig != NULL) {
10            anterior = anterior->sig;
11        }
12        Lista aBorrar = anterior->sig;
13        anterior->sig = NULL;
14        delete aBorrar;
15    }
16 }

```

Complejidad: $O(n)$

Parámetros: Por referencia (&l)

3.4 Liberar Memoria (Destruir Lista)

Propósito: Eliminar todos los nodos y liberar memoria.

```

1 void destruirLista(Lista &l) {
2     while (l != NULL) {
3         Lista aBorrar = l;
4         l = l->sig;
5         delete aBorrar;

```

```

6     }
7 }

```

Complejidad: $O(n)$

Parámetros: Por referencia (&l)

3.5 Inserción Después de un Nodo (insDespues)

Propósito: Insertar elemento después de un nodo dado.

```

1 void insDespues(Lista r, Tipo e) {
2     if (r != NULL) {
3         Lista nuevo = new nodoLista;
4         nuevo->info = e;
5         nuevo->sig = r->sig;
6         r->sig = nuevo;
7     }
8 }

```

Complejidad: $O(1)$

Parámetros: r por valor, e por valor

3.6 Inserción Antes de un Nodo (insAntes)

Propósito: Insertar elemento antes de un nodo dado.

```

1 void insAntes(Lista r, Tipo e) {
2     if (r != NULL) {
3         Lista q = new nodoLista;
4         *q = *r; // Copia superficial
5         r->info = e;
6         r->sig = q;
7     }
8 }

```

Complejidad: $O(1)$

Parámetros: r por valor, e por valor

Advertencia: No usar si 'Tipo' tiene punteros o recursos propios

3.7 Borrar Sucesor de un Nodo (elimSucesor)

Propósito: Eliminar el nodo siguiente a uno dado.

```

1 void elimSucesor(Lista r) {
2     if (r != NULL && r->sig != NULL) {
3         Lista aBorrar = r->sig;
4         r->sig = aBorrar->sig;
5         delete aBorrar;
6     }
7 }

```

Complejidad: $O(1)$

Parámetros: r por valor

3.8 Impresión Recursiva

Propósito: Imprimir lista usando recursión.

```
1 void imprimirListaRec(Lista l) {
2     if (l != NULL) {
3         std::cout << l->info << " -> ";
4         imprimirListaRec(l->sig);
5     } else {
6         std::cout << "NULL" << std::endl;
7     }
8 }
```

Complejidad: $O(n)$

Parámetros: Por valor (Lista l)

3.9 Copia Recursiva

Propósito: Crear una copia independiente de la lista.

```
1 Lista copia(Lista l) {
2     if (l == NULL) return NULL;
3
4     Lista nuevo = new nodoLista;
5     nuevo->info = l->info;
6     nuevo->sig = copia(l->sig);
7     return nuevo;
8 }
```

Complejidad: $O(n)$

Parámetros: Por valor (Lista l)

3.10 Concatenación (Modificando l1)

Propósito: Unir l2 al final de l1 (modifica l1).

```
1 void concat(Lista &l1, Lista l2) {
2     if (l1 == NULL) {
3         l1 = copia(l2);
4     } else {
5         Lista aux = l1;
6         while (aux->sig != NULL) {
7             aux = aux->sig;
8         }
9         aux->sig = copia(l2);
10    }
11 }
```

Complejidad: $O(n + m)$

Parámetros: l1 por referencia, l2 por valor

3.11 Insert Sort (Usando insOrd)

Propósito: Ordenar lista mediante inserción ordenada.

```
1 Lista insertSortSimple(Lista l) {  
2     Lista lres = NULL;  
3     Lista aux = l;  
4     while (aux != NULL) {  
5         insOrd(lres, aux->info);  
6         aux = aux->sig;  
7     }  
8     return lres;  
9 }
```

Complejidad: $O(n^2)$

Parámetros: Por valor (Lista l)

3.12 Invertir Lista (Creando Nuevos Nodos)

Propósito: Crear lista invertida con nuevos nodos.

```
1 Lista invListaNuevosNodos(Lista l) {  
2     Lista l_inv = NULL;  
3     Lista aux = l;  
4     while (aux != NULL) {  
5         insComienzo(l_inv, aux->info);  
6         aux = aux->sig;  
7     }  
8     return l_inv;  
9 }
```

Complejidad: $O(n)$

Parámetros: Por valor (Lista l)

Notas Finales

- Dibujar diagramas de nodos ayuda a entender las operaciones
- Verificar siempre punteros NULL antes de desreferenciar
- La complejidad temporal es crucial para optimización